

BiblioSys — Documentação do Projeto

Instituição: Pontifícia Universidade Católica de Minas Gerais

Disciplina: Algoritmos e Estruturas de Dados III

Professor: Walisson Ferreira de Carvalho

Grupo: 9

Integrantes: Fernando Mucci · Maria Eduarda P. Brito · Luan Oliveira · Luiz Felipe Brandão · Luisa Campanha

Repositório: GitHub — <https://github.com/projetos-puc-minas/TP-AEDS-III-G9.git>

Demonstração: <https://youtu.be/wyZYG1z7cT8>

1. Visão Geral da Fase 3

A Fase 3 do BiblioSys consolida dois relacionamentos N:N já presentes na arquitetura do sistema: Livro ↔ Autor (tabela associativa `livros_autores.bin`) e Livro ↔ Tag (tabela associativa `tags_livros.bin`). Ambas as tabelas possuem chave primária própria (autoincremento), seguem o mesmo padrão de cabeçalho e exclusão lógica com lápide das fases anteriores, e são indexadas por Árvore B+ para suporte a consultas ordenadas e range scan eficiente. O relacionamento é acessível a partir de qualquer uma das duas entidades principais, e a integridade referencial é garantida por verificação nos DAOs responsáveis.

Modelagem de Entidades

Entidade	Arquivo de Dados	Relacionamentos
Livro	<code>livros.bin</code>	N:1 com Editora · N:N com Autor · N:N com Tag
Autores	<code>autores.bin</code>	N:N com Livro (via <code>livros_autores</code>)
Editora	<code>editoras.bin</code>	1:N com Livro
Tag	<code>tags.bin</code>	N:N com Livro (via <code>tags_livros</code>)

Usuarios	usuarios.bin	Autenticação do sistema
LivroAutor	livros_autores.bin	Tabela associativa Livro ↔ Autor
TagsLivros	tags_livros.bin	Tabela associativa Livro ↔ Tag

2. Estrutura do Projeto

```

TP-AEDS-III-G9/
├── src/main/java/
│   ├── Main.java
│   └── src/
│       ├── dao/ # Um DAO por entidade – persistência e indexação
│       │   ├── AutoresDAO.java
│       │   ├── DAOFactory.java
│       │   ├── EditoraDAO.java
│       │   ├── LivroAutorDAO.java ← Tabela associativa Livro ↔ Autor
│       │   ├── LivroDAO.java
│       │   ├── TagDAO.java
│       │   ├── TagsLivrosDAO.java ← Tabela associativa Livro ↔ Tag
│       │   └── UsuarioDAO.java
│       ├── model/ # POJOs com serialização binária
│       │   ├── Autores.java · Editora.java · Livro.java
│       │   └── LivroAutor.java · TagsLivros.java · Tag.java · Usuarios.java
│       ├── server/
│       │   └── ApiServer.java # Servidor HTTP + roteamento REST
│       ├── service/
│       │   └── UsuarioService.java # Autenticação e XOR
│       └── util/
│           ├── Arquivo.java # CRUD genérico com Best-Fit
│           ├── ArvoreBMais.java # Árvore B+ em disco
│           ├── HashExtensivel.java # Hash Extensível (chave int)
│           ├── HashExtensivelLong.java # Hash Extensível (chave long – 1:N)
│           ├── OrdenacaoExterna.java # Intercalação balanceada
│           ├── SerializadorUtil.java # String, String[], int[], ISBN
│           └── DataUtil.java · Indexador.java · Registro.java
├── web/
│   ├── index.html · app.js · style.css # Interface Web (SPA)
├── data/
│   ├── *.bin # Arquivos de dados das entidades
│   └── idx/ # Índices: .idx (B+), .dir.bin + .bkt.bin (Hash)
└── pom.xml # Maven (Java 17 + org.json)

```

3. Relacionamentos N:N

3.1 Livro ↔ Autor (**livros_autores.bin**)

A tabela associativa `LivroAutor` contém os campos `id (int)`, `idLivro (int)` e `idAutor (int)` — todos de tamanho fixo ($3 \times 4 \text{ bytes} = 12 \text{ bytes}$ de dados). O relacionamento é acessível a partir do Livro (quais autores escreveram este livro) e a partir do Autor (quais livros este autor escreveu), por meio de dois índices secundários em Árvore B+.

Índices mantidos por `LivroAutorDAO`:

- `livros_autores_id` (Hash `int` pelo ID) · `livros_autores_por_livro.idx` (B+ com $\text{idLivro} \times \text{MAX_ID} + \text{id}$) · `livros_autores_por_autor.idx` (B+ com $\text{idAutor} \times \text{MAX_ID} + \text{id}$).

3.2 Livro ↔ Tag (`tags_livros.bin`)

A tabela associativa `TagsLivros` contém os campos `id (int)`, `idTag (int)` e `idLivro (int)` — também de tamanho fixo. O relacionamento é acessível tanto do Livro (quais tags ele possui) quanto da Tag (quais livros estão classificados com ela), por índices B+ análogos.

Índices mantidos por `TagsLivrosDAO`:

- `tags_livros_id` (Hash `int` pelo ID) · `tags_livros_por_livro.idx` (B+ com $\text{idLivro} \times \text{MAX_ID} + \text{id}$) · `tags_livros_por_tag.idx` (B+ com $\text{idTag} \times \text{MAX_ID} + \text{id}$).

4. Chave Composta da Tabela Intermediária

Cada tabela associativa possui chave primária inteira própria (autoincremento). A chave composta não é armazenada como campo, mas construída dinamicamente ao inserir/remover entradas nos índices secundários da Árvore B+. A fórmula adotada é `base × MAX_ID + seq` com `MAX_ID = 100_000`:

```
// Constante base para gerar chaves compostas
private static final int MAX_ID = 100_000;

private static long chaveComposta(int base, int seq) {
    return ((long) base * MAX_ID) + seq;
}

// LivroAutorDAO – ao criar vínculo:
indicePorLivro.inserir((int) chaveComposta(idLivro, cr.id), cr.endereco);
indicePorAutor.inserir((int) chaveComposta(idAutor, cr.id), cr.endereco);

// TagsLivrosDAO – ao criar vínculo:
indicePorTag.inserir((int) chaveComposta(idTag, cr.id), cr.endereco);
indicePorLivro.inserir((int) chaveComposta(idLivro, cr.id), cr.endereco);
```

O agrupamento de $\text{base} \times \text{MAX_ID}$ concentra todos os vínculos de uma mesma entidade em uma faixa contígua do espaço de chaves, permitindo que o range scan da Árvore B+ recupere todos os registros associados de forma eficiente, sem varredura sequencial do arquivo.

5. Busca Eficiente por Índice

A busca de todos os vínculos de uma entidade é realizada via range scan nas folhas encadeadas da Árvore B+. O método `buscarPorIndiceSecundario()` calcula $\text{chaveMin} = \text{idFiltro} \times \text{MAX_ID}$ e $\text{chaveMax} = (\text{idFiltro}+1) \times \text{MAX_ID}$ e percorre apenas as entradas nessa faixa, lendo os registros por offset:

```
private List<LivroAutor> buscarPorIndiceSecundario(
    ArvoreBMais indice, int idFiltro, Predicate<LivroAutor> predicado) throws Exception
{
    long[][] pares = indice.listarOrdenado();
    long chaveMin = chaveComposta(idFiltro, 0);
    long chaveMax = chaveComposta(idFiltro + 1, 0);

    List<LivroAutor> resultado = new ArrayList<>();
    for (long[] par : pares) {
        if (par[0] >= chaveMin && par[0] < chaveMax) {
            LivroAutor la = arqLivrosAutores.readByOffset(par[1]);
            if (la != null && predicado.test(la)) resultado.add(la);
        }
    }
    return resultado;
}
```

Para busca direta por PK, o Hash Extensível fornece acesso em $O(1)$ amortizado via `hashId.buscar(id)`. A Árvore B+ também suporta listagem ordenada sem memória extra: `listarOrdenado()` percorre as folhas encadeadas em ordem

crescente e `listarOrdenadoDecrescente()` no sentido inverso, atendendo ao requisito de consulta ordenada sem ordenação em memória principal.

6. Integridade Referencial

Exclusão de Livro

`LivroDAO.excluirLivroEmCascata(id)` remove primeiro todos os vínculos de autores via `livroAutorDAO.excluirAutoresDoLivro(id)` e depois todos os vínculos de tags via `tagsLivrosDAO.excluirTagsDoLivro(id)`, antes de excluir o registro do livro. Cada exclusão de vínculo atualiza todos os índices (B+ e Hash).

Exclusão de Autor

`AutoresDAO.excluirAutor(id)` verifica, antes de prosseguir, se o autor ainda possui livros vinculados via `livroAutorDAO.buscarLivrosDoAutor(id)`. Caso a lista não esteja vazia, uma exceção é lançada, impedindo a exclusão e preservando a consistência.

Exclusão de Tag

`TagDAO` possui referência injetada ao `TagsLivrosDAO`. Antes de excluir uma tag, verifica se existem livros vinculados; caso positivo, remove em cascata todos os vínculos via `excluirLivrosDaTag(id)`.

7. Persistência das Tabelas Associativas

As tabelas `livros_autores.bin` e `tags_livros.bin` seguem exatamente o mesmo padrão de cabeçalho e exclusão lógica das demais entidades do sistema.

Cabeçalho dos Arquivos .bin

Cabeçalho fixo de 16 bytes (`TAM_CABECALHO = 16`):

Offset	Tamanho	Conteúdo
0	4 bytes	Último ID gerado (autoincremento)
4	4 bytes	Total de registros ativos
8	8 bytes	Ponteiro da lista de excluídos (long, sentinela <code>NULO = -1</code>)

Cada registro segue o layout:

```
[ lápide : 1 byte (boolean) ] [ tamanho dos dados : 4 bytes (int) ] [ dados serializados... ]
```

Como todos os campos são **int** (tamanho fixo de 4 bytes cada), os registros das tabelas associativas têm tamanho uniforme de 12 bytes de dados (id + idX + idY), eliminando fragmentação.

Serialização

```
// LivroAutor.toByteArray()
dos.writeInt(id); // 4 bytes
dos.writeInt(idLivro); // 4 bytes
dos.writeInt(idAutor); // 4 bytes

// TagsLivros.toByteArray()
dos.writeInt(id); // 4 bytes
dos.writeInt(idTag); // 4 bytes
dos.writeInt(idLivro); // 4 bytes
```

8. Integração com o CRUD das Entidades Principais

A integração é feita por injeção de dependências via **DAOFactory**. O **LivroDAO** recebe referências ao **LivroAutorDAO** e ao **TagsLivrosDAO** via setters; o **AutoresDAO** recebe referência ao **LivroAutorDAO**; e o **TagDAO** recebe referência ao **TagsLivrosDAO**. O **ApiServer** expõe endpoints dedicados:

Método	Endpoint	Operação
GET	/api/livros-autores	Lista todos os vínculos Livro-Autor
GET	/api/livros-autores?livroId={id}	Lista autores de um livro (range scan B+)
GET	/api/livros-autores?autorId={id}	Lista livros de um autor (range scan B+)
POST	/api/livros-autores	Cria novo vínculo Livro-Autor
DELETE	/api/livros-autores/{id}	Remove vínculo por ID

GET	/api/tags-livros	Lista todos os vínculos Tag-Livro
GET	/api/tags-livros?livroId={id}	Lista tags de um livro (range scan B+)
GET	/api/tags-livros?tagId={id}	Lista livros de uma tag (range scan B+)
POST	/api/tags-livros	Cria novo vínculo Tag-Livro
DELETE	/api/tags-livros/{id}	Remove vínculo por ID

Ao serializar um livro via `livroToJson()`, o servidor consulta automaticamente os dois DAOs associativos para incluir os arrays `autores` e `tags` na resposta JSON.

9. Formulário de Projeto

a) Qual foi o relacionamento N:N escolhido e quais tabelas ele conecta?

Foram implementados dois relacionamentos N:N: Livro ↔ Autor, conectando `Livro` e `Autores` via `livros_autores.bin`; e Livro ↔ Tag, conectando `Livro` e `Tag` via `tags_livros.bin`. Ambas as tabelas já compunham a modelagem do sistema desde a Fase 2 e foram completamente implementadas com CRUD funcional, índices em disco e integridade referencial nesta fase.

b) Qual estrutura de índice foi utilizada (B+ ou Hash Extensível)? Justifique a escolha.

A Árvore B+ foi escolhida como estrutura primária para os índices secundários das tabelas associativas. A justificativa é a necessidade de range scan eficiente: para listar todos os autores de um livro (ou todos os livros de um autor), é preciso recuperar um conjunto de registros cujas chaves estão em uma faixa contígua. A Árvore B+ com folhas encadeadas realiza essa operação em $O(\log n + k)$, onde k é o número de vínculos encontrados, sem varredura completa do arquivo. O Hash Extensível não suporta range scan — é mantido em paralelo apenas para busca direta por PK em $O(1)$ amortizado. A Árvore B+ também oferece listagem ordenada sem memória extra, atendendo ao requisito 4 do enunciado.

c) Como foi implementada a chave composta da tabela intermediária?

As tabelas associativas possuem chave primária própria (int autoincremento). A chave composta não é armazenada como campo no registro, mas construída dinamicamente ao inserir entradas nos índices secundários da Árvore B+. A fórmula é `chaveComposta(base, seq) = base × MAX_ID + seq`, com `MAX_ID = 100_000`. Para Livro-Autor: `indicePorLivro` usa `idLivro` como base e o id do vínculo como seq; `indicePorAutor` usa `idAutor` como base. Isso garante que todos os vínculos de um mesmo livro (ou autor) ocupem uma faixa contígua no espaço de chaves da B+, habilitando o range scan. O mesmo padrão é adotado em `TagsLivrosDAO`.

d) Como é feita a busca eficiente de registros por meio do índice?

A busca por vínculos é feita pelo método `buscarPorIndiceSecundario()`, que chama `listarOrdenado()` na Árvore B+ e percorre apenas as entradas no intervalo `[idFiltro × MAX_ID, (idFiltro+1) × MAX_ID]`. Cada entrada contém o offset do registro no arquivo binário, permitindo leitura direta via `readByOffset()` sem varredura sequencial. Para busca por PK, o Hash Extensível fornece acesso em O(1) amortizado via `hashId.buscar(id)`.

e) Como o sistema trata a integridade referencial (remoção/atualização) entre as tabelas?

Ao excluir um Livro, `LivroDAO.excluirLivroEmCascata(id)` remove primeiro todos os vínculos de autores e tags antes de excluir o livro, atualizando todos os índices. Ao excluir um Autor, `AutoresDAO` verifica se existem livros vinculados e lança exceção caso positivo. Ao excluir uma Tag, `TagDAO` remove em cascata todos os vínculos na tabela `tags_livros.bin`.

f) Como foi organizada a persistência dos dados dessa nova tabela?

As tabelas associativas seguem exatamente o padrão das demais entidades: cabeçalho fixo de 16 bytes e cada registro com layout `[lápide: 1B][tamanho: 4B][dados...]`. A exclusão é lógica: a lápide é marcada como `false` e o endereço do bloco é inserido na lista encadeada de blocos livres para reaproveitamento pela política Best-Fit. Como todos os campos são `int`, os registros têm tamanho uniforme de 12 bytes de dados, eliminando fragmentação.

g) Descreva como o código da tabela intermediária se integra com o CRUD das tabelas principais.

A integração é feita por injeção de dependências via `DAOFactory`. `LivroDAO` recebe `LivroAutorDAO` e `TagsLivrosDAO` por setters; `AutoresDAO` recebe `LivroAutorDAO`; `TagDAO` recebe `TagsLivrosDAO`. O `ApiServer` expõe endpoints dedicados em

`/api/livros-autores` e `/api/tags-livros` para CRUD dos vínculos. Ao serializar um livro em JSON, o servidor enriquece a resposta com arrays `autores` e `tags` consultando os DAOs associativos automaticamente. O frontend SPA consome esses endpoints para exibir e gerenciar os relacionamentos sem uso de console.

h) Descreva como está organizada a estrutura de diretórios e módulos no repositório após esta fase.

```
TP-AEDS-III-G9/
├─ src/main/java/src/
│  ├─ dao/ # Um DAO por entidade – abstrai persistência e índices
│  │  # LivroAutorDAO e TagsLivrosDAO gerenciam as tabelas N:N
│  ├─ model/ # POJOs com toByteArray/fromByteArray (implementam Registro)
│  │  # LivroAutor e TagsLivros são os modelos associativos
│  ├─ server/ # ApiServer: /api/livros-autores e /api/tags-livros
│  ├─ service/ # Regras de negócio desacopladas (autenticação XOR, validações)
│  └─ util/ # Estruturas: Arquivo<T>, ArvoreBMais, HashExtensivel,
│  # HashExtensivelLong, OrdenacaoExterna, SerializadorUtil
├─ web/ # Frontend SPA: index.html, app.js, style.css
├─ data/ # Gerado em runtime: *.bin de dados + idx/ com índices
│  └─ idx/ # livros_autores_por_livro.idx, livros_autores_por_autor.idx,
│  # tags_livros_por_livro.idx, tags_livros_por_tag.idx
├─ docs/ # Documentação das fases
└─ pom.xml # Maven – Java 17, org.json para API REST
```

Arquitetura MVC: `model/` (entidades), `dao/` + `util/` (persistência), `server/` (Controller REST), `web/` (View).

10. Tecnologias Utilizadas

Tecnologia	Finalidade
Java 17	Backend, lógica de persistência e servidor HTTP nativo
Maven + maven-shade-plugin	Build e JAR executável com dependências embutidas
<code>RandomAccessFile</code>	Leitura/escrita direta e posicionada em arquivos binários
<code>org.json</code>	Serialização/deserialização JSON na API REST
HTML5 / CSS3 / JavaScript	Interface web SPA (frontend vanilla)
Árvore B+	Índice dos N:N — range scan e listagem ordenada sem memória extra
Hash Extensível (<code>int</code>)	Busca direta por ID — O(1) amortizado

Hash Extensível (long)	Relacionamento 1:N com range scan por editora
Intercalação Balanceada	Ordenação externa de grande volume com RAM fixa
XOR + Base64	Criptografia manual de senhas com chave derivada do e-mail

Documentação produzida para a Fase 3 — entrega em 26/05/2026. Grupo 9 — AEDs III — PUC Minas.